

VD-Justifications

Can Valid

February 2026

1 Residue

1.1 Motivation

Every VD entry stores its definition as a raw string. A definition typically (i) *invokes* other headwords (concept-words already in the dictionary), and (ii) supplies some connective structure tying those headwords together. The *residue* of a definition is a way to extract and represent this connective structure as a template-with-holes.

1.2 Headwords appearing inside a definition

Fix a tokenization rule for strings (e.g. split into word-tokens plus punctuation). Let $\mathcal{W}$ be the set of headwords currently present in the dictionary.

Definition 1 (Headwords in a definition string). *Given a definition string s , define the set of headwords appearing in s by*

$$\text{Head}(s) := \{h \in \mathcal{W} : h \text{ occurs as a token in } s\}.$$

We often enumerate the unique headwords in s as $\text{Head}(s) = \{h_1, \dots, h_n\}$.

1.3 Residue as a re-insertion function

Intuitively, residue is the operation “remove headwords, keep the in-between text, then make a function that plugs headwords back in.”

Definition 2 (Occurrence decomposition). *Let s be a definition string and suppose the headword occurrences in s (in reading order) are $h_{i_1}, h_{i_2}, \dots, h_{i_k}$, where each $h_{i_j} \in \text{Head}(s)$ and repetitions are allowed. Then s admits a decomposition*

$$s \equiv \sigma_0 h_{i_1} \sigma_1 h_{i_2} \cdots h_{i_k} \sigma_k,$$

where each σ_j is the (possibly empty) substring between headword occurrences.

Definition 3 (Residue). *Let s be a definition string and enumerate the unique headwords $\text{Head}(s) = \{h_1, \dots, h_n\}$. Introduce placeholders $x_1, \dots, x_n$. Define the residue of s to be the n -ary function*

$$\text{Res}_s(x_1, \dots, x_n) := \sigma_0 x_{\phi(i_1)} \sigma_1 x_{\phi(i_2)} \cdots x_{\phi(i_k)} \sigma_k,$$

where $\phi(i_j) \in \{1, \dots, n\}$ maps each headword occurrence h_{i_j} to the index of its corresponding placeholder (i.e. $h_{i_j} = h_{\phi(i_j)}$).

[Reconstruction property] By construction, plugging the original headwords back into the residue recovers the original definition string (up to the chosen tokenization/spacing conventions):

$$\text{Res}_s(h_1, \dots, h_n) \equiv s.$$

Thus residue is a canonical “definition skeleton” together with its headword-holes.

1.4 Meaningfully defined entries

Definition 4 (Meaningfully defined). *A definition string s is meaningfully defined if $\text{Head}(s) = \cdot$. Equivalently, its residue is 0-ary: Res_s is a constant string with no inputs.*

1.5 Residue-based equality (one equality among several)

Residue induces a useful but non-total notion of equivalence between law-strings.

Definition 5 (Residue equality up to placeholder renaming). *Let s, t be two strings with $\text{Head}(s) = \{h_1, \dots, h_n\}$ and $\text{Head}(t) = \{g_1, \dots, g_n\}$. We say s and t are residue-equal up to renaming if there exists a permutation $\pi \in S_n$ such that*

$$\text{Res}_s(x_1, \dots, x_n) \equiv \text{Res}_t(x_{\pi(1)}, \dots, x_{\pi(n)}).$$

[Law sameness via residue] This yields a type of “same law” relation: two laws may be regarded as the same law (in this residue sense) if they involve the same headword signature and their residues are equal or socially/agent-wise similar enough. This sits alongside (and does not replace) other equalities such as literal syntactic equality, entry-identity in the log, and empirical equivalence of predictions.

1.6 Example

If a definition has the form

$$\text{“acceleration is the time – derivative of velocity”}$$

and the headwords in the string are $\{\text{acceleration}, \text{velocity}\}$, then the residue is the binary template

$$\text{Res}(x_1, x_2) = \text{“}x_1 \text{ is the time – derivative of } x_2 \text{”}.$$

Plugging $(x_1, x_2) = (\text{acceleration}, \text{velocity})$ reconstructs the original definition.

2 The Valid Dictionary (VD) Project

2.1 Science has an Audit Problem

The Problem. Physics demands rigorous precision in its mathematics (e.g. $F = ma$), but accepts dangerous ambiguity in its language. We treat scientific theories as static “books” of facts, ignoring the order in which concepts are built. This leads to *Concept Rot*: students attempt to understand high-level abstractions (like “Energy” or “Fields”) without understanding the foundational observations that ground them.

The Solution. The *Valid Dictionary* (VD) is a protocol that redefines a scientific theory as an *append-only log* of semantic transactions. It applies the logic of software version control to the philosophy of science.

2.2 The 3 Core Rules

1. **The Log.** A theory is a sequence of events. You cannot use a “Future Word” to define a “Past Word.”
2. **The Anchor.** Every chain of definitions must eventually hit *Residue*—a direct human sensation or measurement (e.g. a dial reading).
3. **The Loop (The Law Triplet).** A *Physical Law* is not a truth-claim; it is a *closed 3-cycle*. It locks three terms together (like Force, Mass, Acceleration) to create a stable structure.

2.3 The Discovery

When we “audited” Newtonian Mechanics using this protocol, we discovered a hidden hierarchy:

- Newton’s 2nd Law is a *low-degree* loop (close to reality/sensation).
- Newton’s 3rd Law is a *high-degree* loop (requires abstract bookkeeping like “Systems” and “Momentum”).

2.4 The Goal

To create a standard for *auditable science*. By making dependencies explicit, we can improve human education (teaching concepts in the right order) and enable AI to reason about causality without hallucinating.

3 The Valid Dictionary Protocol

To move beyond the vague definition of “scientific theory,” we formalize a theory as a named data structure governed by a strict set of rules. This protocol—the *Valid Dictionary* (VD)—redefines a theory not as a static collection of facts, but as a sequential log of semantic transactions.

3.1 The Dictionary Object

The foundational unit of analysis is the *Dictionary Instance*. We define the dictionary $\mathcal{D}$ not as a function, but as a named authority object consisting of a unique identifier and an ordered history:

$$\mathcal{D} = (\text{Name}, \mathbf{L})$$

Where:

- **Name:** A unique string identifier (e.g. `Newtonian.Mechanics_v1.0`) that establishes the scope of authority.
- **L:** The *Entry-Log*, an ordered, append-only sequence of entries $(E_n)_{n \geq 0}$.

The “state” of a theory is not timeless; it is path-dependent. The state of the dictionary at index n , denoted $\mathcal{D}^{(n)}$, is the cumulative result of all entries E_0 through E_n .

Consequently, the “definition” of any word w is not a single output string, but the complete history of transactions involving w in the log.

3.2 The Entry Structure

Each transaction in the log is a tuple representing a definitional event:

$$E_n = (w, \delta)$$

Components:

- w (*Headword*): The term being defined.
- δ (*Definition String*): The literal syntactic output.

Crucially, the VD distinguishes between *Syntactic Definition* and *Semantic Residue*:

- **Syntactic Definition (δ):** A string defining w in terms of other words in the log (e.g. “`Net_Force := Mass × Acceleration`”).
- **Residue (Γ):** A special class of definition that points outside the log to a human-executable procedure or recognition pattern (e.g. “The sensation of muscle tension” or “The reading on a calibrated dial”). Residue is the mechanism by which the abstract log grounds itself in physical reality.

3.3 The Structure of Physical Law: The Triplet

In standard discourse, a physical law is often treated as a sentence or equation. In the VD protocol, a *Physical Law* is identified as a specific topological structure within the log: a closed 3-cycle of mutually dependent entries.

We term this structure the *Law Triplet*. It rigidly assigns roles to the three terms involved to ensure logical stability without circular collapse.

The Roles

The Anchor Term (w_{Anc}):

- **Constraint:** Must be previously defined (exists at index $k < n$) and must carry traceable Residue.
- **Function:** Serves as the structural bridge between the new law and the existing log. It “donates” meaning to the cycle, preventing the law from being purely tautological.

The Cyclic Terms (w_{Cyc1}, w_{Cyc2}):

- **Constraint:** Must not be previously defined (or are treated as empty slots).
- **Function:** Acquire meaning solely through their structural relationship with the Anchor Term.

The Topology

A Law Triplet is formed by three sequential entries (E_A, E_B, E_C) that lock these terms together:

$$w_{Anc} := f(w_{Cyc1}, w_{Cyc2})$$

$$w_{Cyc1} := f(w_{Anc}, w_{Cyc2})$$

$$w_{Cyc2} := f(w_{Anc}, w_{Cyc1})$$

This structure formalizes the *constructive circularity* of physics. The law does not define the Cyclic Terms in isolation; it defines the relations between them, anchored by w_{Anc} .

3.4 Governance Rules

To ensure the integrity of the audit trail, the VD enforces three strict constraints:

Rule 1: The Law of Linear Provenance A definition E_n is valid if and only if every term used in its definition string δ exists at an index $k < n$, or is a Cyclic Term introduced simultaneously within a Law Triplet.

Rule 2: The Unique-Anchor Constraint To prevent semantic drift, a specific headword w may serve as the Anchor Term for only one Law Triplet within a single dictionary instance.

This forces conceptual disambiguation. If one wishes to define a “Force” with structural behavior different from Newton’s, one must fork the log or introduce a new term (e.g. `Relativistic_Force`) rather than overloading the existing anchor.

Rule 3: The Residue Requirement A dictionary instance is considered *Valid* if and only if every Law Triplet traces its Anchor Term back to a Residue (Γ).

4 Introduction

Formal understanding of the phenomenon of “scientific theories” and their role in science remains either vague or incomplete. While physicists demand rigorous precision in their mathematics, the *theory* itself—the vehicle that carries these equations—is often treated as an intuitive, unstructured object. We aim to rigorously identify the mathematical nature and behaviour of scientific theories by analysing the provenance of their language.

In doing so, we developed a protocol of rules—the *Valid Dictionary* (VD)—that enables scientific theories to be expressed explicitly and auditably. The protocol identifies the theory-specific dictionary as a critical structural feature of any theory, managing it not as a static list, but as an *audit trail* of definition entries. By encoding theories into this common framework, we can trace the lineage of concepts and interrogate significant linguistic features of the laws. Importantly, a *Physical Law* is identified not as a standalone equation, but as a structural feature of this log: a triplet of interdependent definitions.

We apply this framework by distinguishing between abstract theories (such as Newtonian Mechanics), which we call *Theory Classes*, and the specific material used to teach them, which we call *Theory Instances*. This distinction is central to our methodology. Just as a mechanic studies the abstract engineering of a “Car” by examining a specific vehicle on a lift, we study the abstract nature of a Theory Class by auditing a specific Theory Instance. While every specific car carries a unique history of repairs and modifications—analogous to the specific diagrams and explanations added by a lecturer—the underlying engine (the Dictionary structure) remains accessible for structural analysis.

We used the protocol to encode a simplified version of Newtonian Mechanics, enabling a systematised discussion of its internal architecture. The results demonstrate that by making these hidden structures explicit, we can not only audit the logical consistency of a theory, but also provide concrete methods to improve the clarity and quality of physics education.